Quicx with an Adaptive Pool

Combining allocator research in a task queue that lives on a fixed memory budget

Dimitar Anastasov

November 2026 · branch `adaptive-pool`, apool v2

Abstract

Quicx is a single-threaded task queue daemon written in C that never uses more memory than one budget fixed at start-up. Until version 1.0.4 that budget was split between block sizes by fixed percentages (PMAD), so most of it sat unused whenever the traffic did not match the split. This report describes *apool v2*, an allocator whose 16 KiB pages move between block sizes as the traffic changes. It combines five ideas from the allocator literature, each behind a switch so it could be measured on its own, with two changes of my own. On an Apple M4 Pro with the default 8 MiB budget, the daemon holds 46,593 tasks of 100 B instead of 655, and 30,947 generated nginx log lines instead of 1,433. After a burst drains it keeps 2,080 KB of physical memory instead of 9,200 KB. Throughput and latency are unchanged within the run-to-run noise. The end-to-end tests also found a liveness bug in the released 1.0.4: once its queue is full of 16 B or 32 B tasks, a worker that connects afterwards can never register, so the queue can never be drained. One of the five ideas, fullest-page-first from Hoard, gave no benefit on queue traffic and is switched off. A second machine, a 2011 Intel Core i3 PC running Ubuntu Linux, reproduces every capacity number exactly, the 1.0.4 deadlock and the cost of purging (Section 9). All numbers come from raw logs in `results/`.

1 Introduction

A task queue sits between *producers*, which submit work, and *workers*, which do it. Quicx keeps every queued task in memory, and it promises never to grow beyond a configured budget: when the budget is full it refuses new work with `ERROR 0x01` instead of growing. That promise is only useful if the budget can actually be filled. A static split by block size cannot guarantee this. With the default PMAD configuration, a queue of 100 B tasks is refused after 655 tasks, while almost all of the 8 MiB is still free in other size classes.

The contribution of this work is engineering and measurement, not a new algorithm. Every mechanism used here exists in published allocators, and Section 4 names the source of each. What is new for Quicx is:

- an allocator that keeps the hard budget of the old design but lets pages move between sizes;
- five published ideas combined in it, each behind a flag, with a measured benefit and cost for each;
- a separate *control pool* for connections and workers, which removes a liveness failure that the static design has;
- tests and benchmarks for every feature, run on real hardware, with the raw output kept.

2 System overview

Figure 1 shows the whole daemon. One event loop (kqueue on macOS, epoll on Linux) wakes up when a socket is readable. `server.c` reads and checks each frame: a 6-byte header followed by a body of at most 1,024 B. `daemon.c` decides what each message means, and `dispacher.c` pairs waiting tasks with idle workers. The budget is split once at start-up by `quicx_init` (`quicx.c`): 3 pages (49,280 B) form the control pool for 1,024 connections and 1,024 workers, and the rest (8,339,328 B, 508 pages) is the task pool.

Both diagrams follow the C4 model's notation rules and review checklist [5]: a title that names the diagram type and scope, a legend for every colour, shape, border and line style, a type or technology for every element, and a label and one direction for every arrow.

A task is *one block*: the 64 B `Task` record followed by the payload. `server.c` allocates the block before it reads the payload, so the payload is read straight into its final place. The dispatcher frees the block as soon as the `TASK` message is sent, which makes delivery at-most-once.

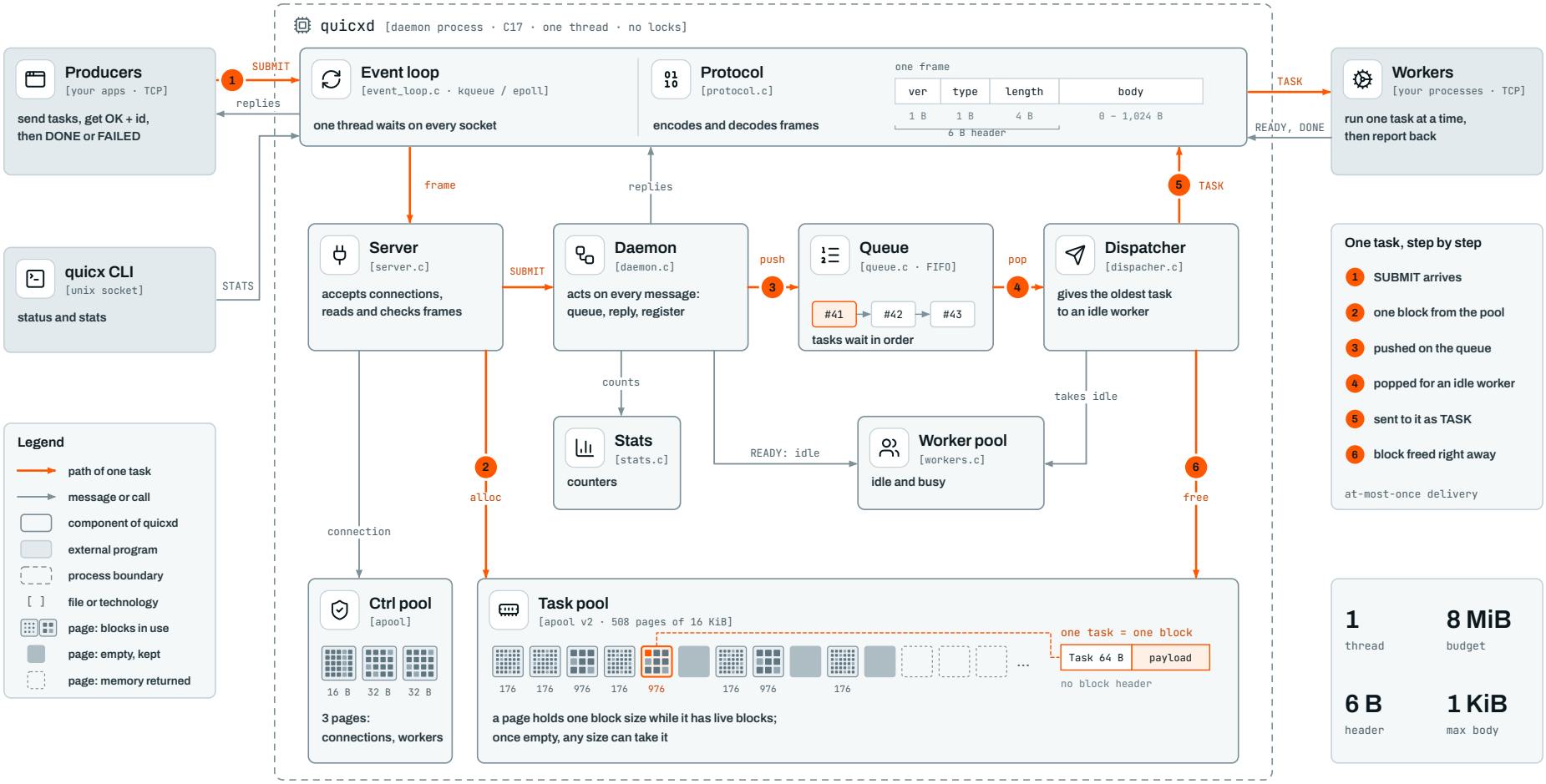


Figure 1: Component diagram of Quicx with apool v2. Every element has a name, a `[file or technology]` tag and a one-line description; every arrow points from the sender and is labelled; the legend explains every fill, border and line style. Orange marks the path of one task, steps 1–6. Icons: Lucide (ISC licence). A dark version for slides and the website is in `figures/`.

3 The allocator

Figure 2 shows apool v2 (`quicx/src/apool.c`, 341 lines). A pool is one `mmap` that is never grown or moved. It is cut into 16 KiB pages, followed by one 32 B record per page. A page that is in use belongs to one block size (68 sizes, 16 to 1,088 B in steps of 16) and one lifetime class. Blocks carry no header. The page of a block is found by dividing its offset by 16,384, and the page's record holds everything else. Inside a page, freed blocks form a list through their own first 8 bytes, and blocks that were never used are handed out from a bump offset. When the last block of a page is freed, the page leaves its size and goes back to a shared free list, where *any* size can take it next. That is the reshaping.

4 Features taken from papers

Table 1 lists every feature, where it came from, what it buys and where it is in the code. The flags are set per pool before the first allocation, and `APOOL_F_DEFAULT` is `RESERVE | HARDEN | PURGE`. Each feature can be switched in the `[allocator]` section of `quicx.conf` (`reserve`, `hardening`, `purge`, `fullest_first`, plus `warm_pages` and `reserve_pages`); the file in the repository lists every key with its default, and `pool_config.c` reads them into one `PoolConfig`.

Feature	Source	Benefit	Where (file, function)	How it works	Default
Pages move between sizes (the core)	slab allocator [4]; mimalloc pages [9]; memcached [12]	the same budget serves whatever sizes arrive	`apool.c`: `take_page` (l. 126), `release_page` (l. 157)	a size takes a free page only when it has no room; a page whose last block is freed returns to the shared list	always
Reserve	Linux zone watermarks [8]; memcached per-class minimum [12]	one size cannot lock every other size out	`apool.c`: `take_page` (l. 128), `apool_init` (l. 208)	with $\leq n/64$ free pages (7 of 508), only a size that owns no page may take one	on
Hardening	mimalloc secure mode [9]	corruption and double frees are detected, not silent	`apool.c`: `encode`/`decode` (l. 31), `apool_alloc_hint` (l. 252), `apool_free` (l. 306)	free-list links are XOR-ed with a per-pool key; a decoded link must point into the same page; a block that looks freed triggers a bounded walk of its page's list	on
Purge with a warm list	jemalloc [7]; mimalloc [9]; `madvise` [2, 10]	memory goes back to the OS after a burst	`apool.c`: `release_page` (l. 165), `purge`/`unpurge` (l. 106)	empty pages are reused newest first; beyond $n/8$ empty pages (63 of 508) the coldest is purged (`MADV_FREE_REUSABLE` on macOS, `MADV_DONTNEED` on Linux)	on
Lifetime classes	LLAMA [11]	a few long-lived blocks cannot pin pages full of short-lived ones	`apool.c`: `apool_alloc_hint`, `group_head` (l. 59)	a caller marks a block as long-lived; such blocks get their own pages	available, unused
Fullest page first	Hoard [3]	meant to let emptier pages drain so they can be reshaped	`apool.c`: `group_of`, `regroup` (l. 64–76)	four fullness groups per size; allocation uses the fullest page	off
One block per task	own change	one allocation and one free per task; no record/payload split	`server.c`: `handle_readable` (l. 163); `dispacher.c` (l. 24)	the payload sits right behind the 64 B `Task` record	always
Control pool	own change; same principle as reserved memory for forward progress [6]	a full task pool cannot stop workers from registering	`quicx.c`: `quicx_init` (l. 12–22); `server.c` l. 81; `daemon.c` l. 42	Connections and Workers come from a separate 3-page pool sized for 1,024 of each	always

Table 1: Features of apool v2, their sources, benefits and places in the code. Line numbers refer to the `adaptive-pool` worktree on 8 Nov 2026.

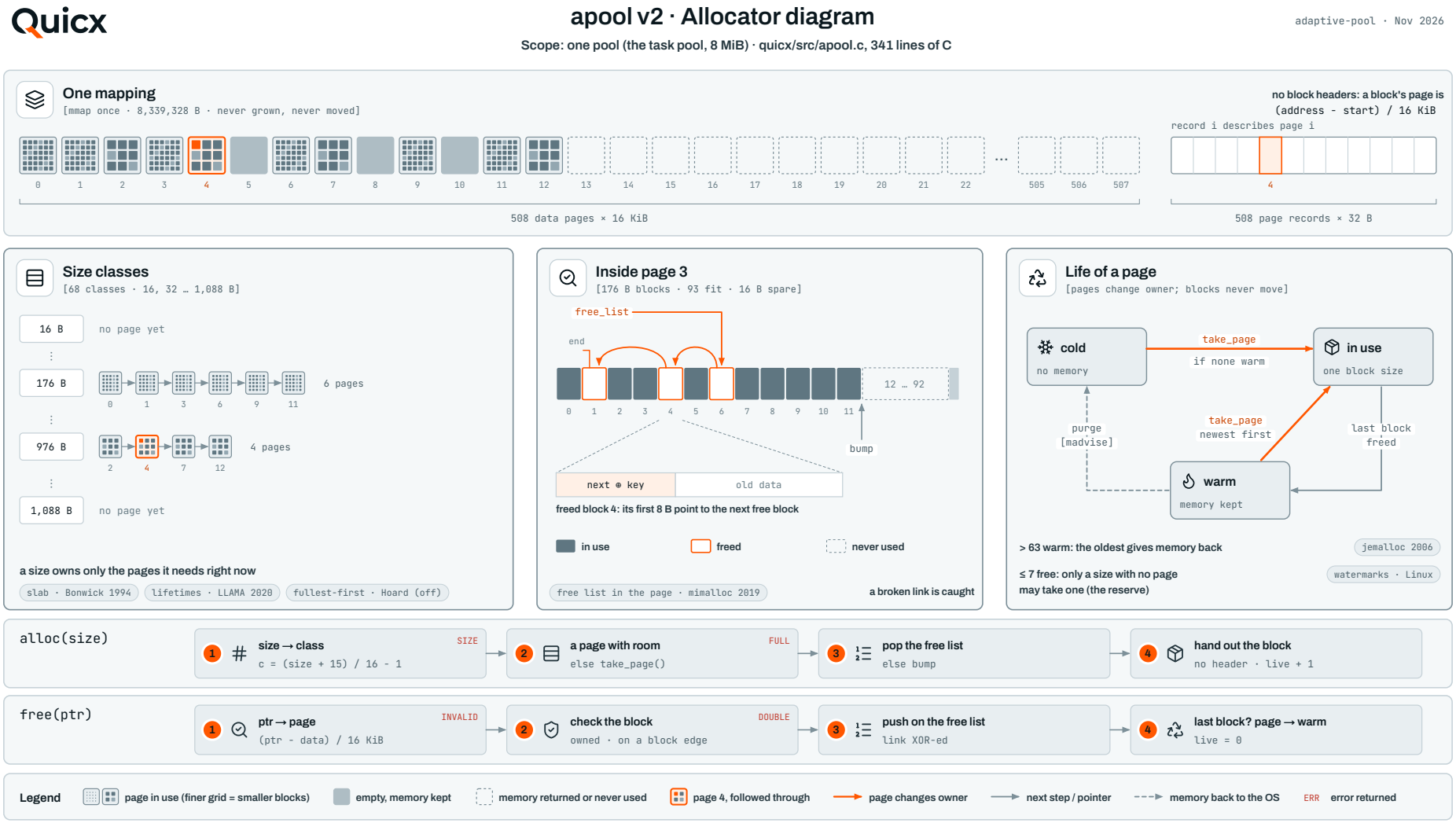


Figure 2: apool v2: the one mapping with its page records, the size classes and the pages each one owns, the inside of one page with its free list, the life of a page, and the alloc and free paths with the error each step can return. Page 4 is followed through every part. Grey tags name the paper each idea came from (Table 1).

Pages move between sizes. This is the slab idea [4]: memory is managed in pages, and each page holds blocks of one size. Like mimalloc's pages [9], an empty page is not kept by its size but returned for any size to reuse. memcached reaches a similar effect by moving 1 MB pages between its classes in the background [12]. apool only ever moves *empty* pages, because a queue cannot evict a task the way a cache can evict an item.

Reserve. Linux keeps a few pages per zone below a watermark for allocations that must succeed [8]. apool keeps the last $n/64$ free pages for sizes that own no page yet. Without it, a flood of one size takes every page and the next size is refused completely.

Hardening. Following mimalloc's secure mode [9], the address stored inside a free block is XOR-ed with a random per-pool key. A block overwritten after it was freed decodes to an address outside its page and is detected. A second free of the same block is found by walking that page's free list, which happens only when the block's first word looks like a link. The first word is cleared on every allocation, so a live block almost never looks like one.

Purge with a warm list. jemalloc and mimalloc give unused memory back to the operating system [7, 9]. Doing this on every empty page would put a system call on the hot path. apool keeps the $n/8$ most recently emptied pages *warm* and purges only beyond that, oldest first.

Lifetime classes. LLAMA predicts object lifetimes and groups objects by them [11]. apool takes the simple form of the idea: the caller passes a hint, and long-lived blocks get pages of their own. The daemon passes no hint today, because every task is freed in arrival order, so the feature is ready for redelivery, priorities or delayed tasks.

Fullest page first: a negative result. Hoard sorts pages into fullness groups and allocates from the fuller ones so that the emptier ones drain [3]. On queue traffic this did not help. Tasks are freed almost in arrival order, so pages already drain on their own, and the extra list moves cost time (Table 7). The feature stays in the code behind its flag and is off by default.

5 Method

Machines. Table 2 lists the machines. Both machines ran the same scripts on 8 November 2026. Each built its own binaries with its own compiler. The Linux server is a much older and slower machine, which is useful: an effect that shows on both is not a property of one CPU or one kernel (Section 9).

	Mac	Server
CPU	Apple M4 Pro	Intel(R) Core(TM) i3-2100 CPU @ 3.10GHz
Cores	12 (8 performance + 4 efficiency)	2 cores, 4 threads
RAM	24 GiB	7.6 GiB
OS	macOS 26.5.1	Ubuntu 24.04.3 LTS
Kernel	Darwin 25.5.0	Linux 6.8.0-147-generic
OS page size	16,384 B	4,096 B
CPU governor	—	schedutil
Compiler	Apple clang version 17.0.0 (clang-1700.6.3.2)	cc (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0
Open-loop rate	40,000 tasks/s	9,000 tasks/s

Table 2: Machines (from `results/*/machine.txt`).

Versions. **1.0.4** is the released version (tag `v1.0.4`, static PMAD pool [1]). **v1** is the first adaptive pool from before the papers (commit `e7d4cdf`). **v2** is the current `adaptive-pool` worktree. All three daemons are built by `scripts/build.sh` with the same compiler and the release flags (`-std=c17 -O2`, as the CI release build uses). In the allocator-only benchmark, "apool v1" is the v2 code with every flag off and two blocks per task, which is how v1 behaved.

Repetitions. The allocator benchmarks run 5 times, the page-size sweep 3 times and the end-to-end capacity runs 3 times. The throughput and latency runs are 5 runs per version, interleaved (1.0.4, v1, v2, 1.0.4, …) so that slow drift affects every version alike. Timings are reported as the median with the minimum and maximum in brackets. Counts that should not change between runs are printed once; a † would mark a count that changed, and none did.

Memory. Physical memory is `phys_footprint` on macOS (what `MADV_FREE_REUSABLE` lowers and what Activity Monitor shows), read with `vmmap` for the daemon and with `task_info` inside `bench_purge`. On Linux it is the resident set size from `/proc`.

Load. The end-to-end clients are the C programs of the `bench` branch (commit `bfff3d0`, copied to `scripts/bench/`): 8 workers and 2 producer processes with 4 connections each, on the same machine as the daemon, over loopback. The payloads are nginx access-log lines generated by `workload.h`. Closed loop keeps 32 tasks in flight per connection. Open loop sends at a fixed rate (Mac 40,000 tasks/s, Server 9,000 tasks/s) and measures latency from the intended send time. Protocol errors and dropped connections reported by the producers, over all runs: Mac: 0, Server: 0.

6 Tests

Every feature has at least one test that shows it works. The tests are: `test_apool` (50 checks, `quicx/src/tests/test_apool.c`), `test_wire` (28 checks), and the end-to-end checks in `scripts/e2e_tests.py`, which drive a real daemon over TCP. Tables 3 and 4 show the results.

Test	What it proves	Feature	Mac	Server
`test_apool`: random stress, daemon features	random sizes and order, 1 in 8 blocks long-lived, with the daemon's flags: no overlap, all memory returns, pages reshaped and purged	core, reserve, hardening, purge, lifetimes together	9/9 pass	9/9 pass
`test_apool`: random stress, all features	the same with every allocator feature on at once	all six allocator features together	9/9 pass	9/9 pass
`test_apool`: reshape	the same pages serve 64 B, then 1,024 B blocks	pages move between sizes	4/4 pass	4/4 pass
`test_apool`: reserve	a flood leaves the reserve; a new size still gets one page	reserve	3/3 pass	3/3 pass
`test_apool`: fullest page first	allocation picks the fuller page	fullest-first	2/2 pass	2/2 pass
`test_apool`: lifetime classes	short- and long-lived blocks never share a page	lifetime classes	2/2 pass	2/2 pass
`test_apool`: double free	a second free is refused and counted	hardening	6/6 pass	6/6 pass
`test_apool`: corrupted free list	an overwritten link is detected and cut	hardening	4/4 pass	4/4 pass
`test_apool`: purge	pages beyond the warm set are purged and work again	purge	2/2 pass	2/2 pass
`test_apool`: invalid input	bad sizes and bad pointers are refused	core	9/9 pass	9/9 pass
`test_wire`	frames with payloads reach the socket intact	protocol fix (v1.0.4)	28/28 pass	28/28 pass

Table 3: Unit tests of the allocator and the wire format (`results/*/test_apool.txt`, `test_wire.txt`).

			Mac		Server	
Test	What it checks	Feature	1.0.4	v2	1.0.4	v2
roundtrip	payloads of 0–1,019 B arrive byte-exact; DONE reaches the producer	one block per task	pass	pass	pass	pass
oversize	a body over 1,024 B gets ERROR 0x03 and the connection closes	protocol	pass	pass	pass	pass
refusal-100	a full queue answers ERROR 0x01; the stream stays in sync	drain on refusal	pass	pass	pass	pass
liveness-16	after the queue fills with 16 B tasks, a new worker connects and drains all	control pool	**fail**	pass	**fail**	pass
liveness-32	the same with 32 B tasks	control pool	**fail**	pass	**fail**	pass
liveness-100	the same with 100 B tasks	control pool	pass	pass	pass	pass
liveness-900	the same with 900 B tasks	control pool	pass	pass	pass	pass
stale	DONE from a connection that never sent READY does not crash	Connection reset	pass	pass	pass	pass

Table 4: End-to-end tests against real daemons (`results/*/e2e_*.txt`). v1 passes the same checks as v2. Each test starts its own daemon.

The checks that failed, with the reason each test printed:

- Mac, 1.0.4, `liveness-16/late-worker-drains-all`: drained 0/5240; the daemon closed the worker's connection (no memory for it)

- Mac, 1.0.4, `liveness-16/queue-usable-after`: type 3
- Mac, 1.0.4, `liveness-32/late-worker-drains-all`: drained 0/3494; READY refused: 'worker pool full'
- Mac, 1.0.4, `liveness-32/queue-usable-after`: type 3
- Server, 1.0.4, `liveness-16/late-worker-drains-all`: drained 0/5240; the daemon closed the worker's connection (no memory for it)
- Server, 1.0.4, `liveness-16/queue-usable-after`: type 3
- Server, 1.0.4, `liveness-32/late-worker-drains-all`: drained 0/3494; READY refused: 'worker pool full'
- Server, 1.0.4, `liveness-32/queue-usable-after`: type 3

6.1 All features together

Each feature is tested on its own above, but in the daemon they all run at the same time. Two checks show that they work together (Table 5). The allocator stress test makes 2 million random allocations and frees of 1 to 1,088 B, marks 1 in 8 blocks as long-lived and checks every byte of every block. It runs once with the daemon's features and once with every allocator feature on, fullest-first included. The daemon was also run with every feature on (`fullest_first = on` in `quicx.conf`; the test build `quicxd-v2-all` makes that its default with `-DQUICX_POOL_FLAGS=APOOL_F_ALL`, `pool_config.c`) and put through the same end-to-end tests and capacity runs. Nothing breaks and nothing changes: the counts are identical and memory still goes back after every burst.

In the daemon as shipped, six of the eight features run together all the time: pages moving between sizes, the reserve, hardening, purge, one block per task and the control pool. Fullest-first is off because it did not help (Section 4). Lifetime classes work in the allocator, but the daemon passes no hint yet.

	Mac		Server	
	v2	all features on	v2	all features on
Allocator stress test, checks passed	9/9	9/9	9/9	9/9
End-to-end checks passed	27/27	27/27	27/27	27/27
100 B tasks that fit	46,593	46,593	46,593	46,593
nginx lines that fit	30,947	30,947	30,947	30,947
Memory after the last burst (KB)	2,080 (2,080–2,096)	2,096	2,736 (2,736–2,740)	2,740 (2,736–2,740)

Table 5: The features working together: v2 as shipped against v2 with every allocator feature on (`results/*/test_apool.txt`, `e2e_v2-all.txt`, `capacity_v2-all_*.txt`).

7 Benchmarks

Table 6 lists every benchmark, what it measures, which feature it tests and how to run it. The code is in the named files; it is not repeated in the tables.

Benchmark	What it measures	Feature / source	File, command
capacity	100 B tasks that fit in 8 MiB	core [4, 9]	`bench_apool.c` `w_capacity`; `bench_apool`
phases	tasks that fit as the size changes 100 → 900 → 40 → 300 B on one pool	core (reshaping)	`w_phases`
starve	900 B tasks that still fit after 40 B tasks fill the pool	reserve [8]	`w_starve`
pinned	900 B tasks that fit after 100 B tasks drain but 1 in 20 stays	lifetime classes [11]	`w_pin`
depth	deepest mixed-size queue (FIFO, 8 out of order, random) that never fails	fullest-first [3], fragmentation	`w_depth`
ns per task	alloc + free per task in a 1,000-deep FIFO churn	hardening [9], one block per task	`w_speed`
page size	the above for 4–64 KiB pages	page size choice	`bench_apool_ps*` (`-DAPOOL_PAGE_SIZE`)
purge	physical memory full and after a drain; cost of pool-sized bursts; purges during steady churn	purge [7, 9]	`bench_purge.c`; `bench_purge`
daemon capacity	tasks a real daemon accepts per phase (100, 900, 40, 300 B, nginx lines) and its physical memory	core, control pool, purge	`scripts/capacity.py` with the C client `fill.c`
throughput	closed-loop tasks/s, whole daemon	everything, end to end	`scripts/bench/run.sh -w 8 -p 2 -c 4 -i 32 -d 10 -u 2`
latency	p50 / p99 / p99.9 at a fixed rate	everything, end to end	`run.sh ... -r RATE`

Table 6: Benchmark catalogue. `scripts/run_all.sh MACHINE` runs all of them; `scripts/make_tables.py` builds the tables below from the raw output.

8 Results

8.1 Allocator ablation

Table 7 compares PMAD, v1, v2 and v2 with one feature switched off (or on), in one 8 MiB pool, on the Mac (5 runs).

	tasks that fit (capacity, then phases)						deepest queue		
Allocator	100 B	900 B	40 B	300 B	starve	pinned	FIFO	random	ns/task
PMAD, default config	655	180	1,310	262	180	180	5,312	5,312	13.3 (13.1–13.3)
apool v1	47,450	8,143	74,679	22,419	0	0	30,720	30,208	24.9 (23.7–25.9)
apool v2	46,872	8,064	73,584	22,176	16	7,648	29,952	29,184	14.8 (14.5–15.0)
v2 + fullest first	46,872	8,064	73,584	22,176	16	7,648	29,696	29,184	20.0 (19.7–20.1)
v2 − reserve	47,523	8,176	74,606	22,484	0	7,760	30,208	29,696	14.7 (14.5–15.0)
v2 − hardening	46,872	8,064	73,584	22,176	16	7,648	29,952	29,184	13.8 (13.7–13.9)
v2 − lifetime hints	46,872	8,064	73,584	22,176	16	16	29,952	29,184	14.7 (14.7–15.0)
v2 − one block/task	46,848	8,024	73,656	22,101	17	7,616	30,208	29,952	24.5 (24.3–24.9)

Table 7: Allocator-only ablation on the Mac (`results/mac/bench_apool_*.txt`). ns/task: median and range of the runs; the other columns were identical in every run.

What each line shows:

- **Core.** v2 holds 46,872 tasks of 100 B where PMAD holds 655, about 71 times more, and it follows every change of size.
- **Reserve.** Without it, a size arriving after a flood gets nothing (0). With it, the new size gets one page (16 tasks of 900 B). This costs 651 tasks of capacity at 100 B (47,523 → 46,872, 1.4%).
- **Lifetime hints.** With hints, 7,648 tasks of 900 B fit after the pinning pattern; without them, 16.
- **Hardening** costs about 1 ns per task (13.8 → 14.8).
- **One block per task** saves about 10 ns per task (24.5 → 14.8).
- **Fullest-first** lowers the deepest FIFO queue (29,952 → 29,696) and costs about 5 ns per task.

The server's version of this table is in Appendix A.

8.2 Purge

		full (KB)	after drain (KB)	burst ns/block	churn ns/task	churn purges
Mac	purge on (default)	8,160 (8,160–8,176)	1,104 (1,104–1,120)	102.1 (44.9–125.1)	12.6 (12.5–13.8)	0
Mac	purge off	8,096 (8,096–8,112)	8,096 (8,096–8,112)	8.7 (8.5–8.8)	12.4 (12.4–12.6)	0
Server	purge on (default)	8,336	1,280	584.7 (584.0–629.8)	62.6 (62.5–63.2)	0
Server	purge off	8,080	8,080	51.1 (50.3–52.0)	62.6 (62.5–63.2)	0

Table 8: `bench_purge`: physical memory of the process above its baseline, with the pool full of 964 B blocks and after all are freed; time per block over 40 pool-sized bursts; time per task in steady churn (1,000 in flight). Memory measure: Mac: `phys_footprint`, Server: `rss`.

Purging works as intended. After a drain the pool holds 1,104 KB above baseline instead of 8,096 KB, and steady churn never purges (0 purges, same time per task). The cost is on bursts that fill and empty the *whole* pool again and again. Each such burst purges 441 pages and brings them back, which costs 102 ns per block instead of 8.7 (Section 10). The server shows the same trade-off at its own speed: 584.7 instead of 51.1 ns per block, again about 11 times more, and 1,280 KB instead of 8,080 KB after a drain.

8.3 Page size

	Mac				Server			
Page	100 B	starve	FIFO depth	ns/task	100 B	starve	FIFO depth	ns/task
4 KiB	46,023	4	30,464	22.2 (21.7–22.3)	46,023	4	30,464	78.3
8 KiB	46,230	8	30,464	18.4 (18.2–18.4)	46,230	8	30,464	75.2 (75.2–75.3)
16 KiB	46,872	16	29,952	14.9 (14.7–15.0)	46,872	16	29,952	72.9 (72.7–72.9)
32 KiB	46,872	33	27,392	14.2 (14.1–14.4)	46,872	33	27,392	72.3
64 KiB	46,872	67	23,680	14.1 (14.0–14.2)	46,872	67	23,680	72.7 (72.7–73.3)

Table 9: apool v2 with other page sizes (`results/*/bench_ps*_*.txt`).

Smaller pages make every page change cost relatively more (22.2 ns per task at 4 KiB) and give a new size fewer blocks from the reserve. Larger pages waste more space at the end of each size's last page, so the deepest mixed queue falls from 29,952 (16 KiB) to 23,680 (64 KiB). 16 KiB is the best balance of the sizes tried, and it is also the OS page size on Apple Silicon, so one apool page is exactly one OS page to purge. The server gives the same counts and the same order of times (78.3 ns per task at 4 KiB, 72.9 at 16 KiB, no gain beyond).

8.4 Whole daemon: capacity and memory

	tasks accepted before ERROR 0x01					physical memory (KB)			
Version	100 B	900 B	40 B	300 B	nginx	idle	full	after drain	all drained
1.0.4	655	180	1,310	262	1,433	9,200	9,200	9,200	yes
v1	47,158	8,092	74,240	22,272	31,708	1,040	9,168 (9,168–9,184)	9,184 (9,184–9,200)	yes
v2	46,593	8,016	73,146	22,044	30,947	1,056	9,088 (9,088–9,104)	2,080 (2,080–2,096)	yes
v2, all features on	46,593	8,016	73,146	22,044	30,947	1,072	9,104	2,096	yes

Table 10: One daemon per version, phases in this order, 8 MiB budget, Mac (`results/mac/capacity_*.txt`, 3 runs). "after drain" is after the last phase. Memory measure: `phys_footprint`.

The daemon numbers match the allocator benchmark, minus the 3 pages that now belong to the control pool. 1.0.4 holds its full 9,200 KB from the moment it starts, because PMAD writes a header into every block at start-up. v1 starts small but keeps whatever a burst touched. v2 starts at 1,056 KB and returns to 2,080 KB after every burst. The server repeats the pattern in resident memory (Appendix A): 1.0.4 holds 9,928 KB throughout, v1 grows from 1,784 to 9,916 KB and stays there, and v2 grows to 9,816 KB during the burst and returns to 2,736 KB.

8.5 Throughput and latency

		closed loop	open loop at a fixed rate		
Machine	Version	tasks/s	p50 µs	p99 µs	p99.9 µs
Mac	1.0.4	82.4k (79.4–86.4)	47 (45–49)	111 (89–155)	252 (131–675)
	v1	80.8k (77.6–85.0)	47 (45–48)	110 (90–142)	163 (129–2,192)
	v2	81.9k (77.8–85.4)	49 (45–54)	98 (92–395)	143 (133–4,470)
Server	1.0.4	19.0k (18.4–19.0)	128 (113–146)	162 (127–195)	207 (198–1,479)
	v1	19.0k	147 (115–157)	200 (132–561)	214 (150–4,296)
	v2	19.0k (18.8–19.2)	123 (114–164)	198 (126–202)	208 (137–1,031)

Table 11: Closed-loop throughput and open-loop latency (rate: Mac 40,000 tasks/s, Server 9,000 tasks/s), median and range of 5 interleaved runs per version (`results/*/closed_*.txt`, `open_*.txt`).

The three versions are within each other's run-to-run range on every column. A task's round trip through the daemon takes about 12 µs, and the allocator's share of that, about 15 ns, is below the noise. On the server all three versions reach 19.0k tasks/s. There, eleven processes (the daemon, 8 workers and 2 producers) share 4 hardware threads, so the CPU is the limit and the three versions cannot be told apart. The new allocator did not make Quicx faster or slower on either machine; it changed how much fits and how much memory stays in use.

9 Server runs

The second machine is `da`, a desktop PC from 2011 (Intel Core i3-2100, 2 cores and 4 threads, 7.6 GiB, Ubuntu 24.04, gcc 13.3), reached over Tailscale. `scripts/run_server.sh` copied the source to `~/quicx-report/` on it, built everything there with its own compiler, ran the same suite unattended (about 14 minutes) and copied `results/server/` back. Its results stand next to the Mac's in every table above and in Appendix A. What the second machine adds:

- **Capacity is a property of the allocator, not of the machine.** Every count of tasks that fit is the same on both machines, in the allocator benchmark and in the whole daemon, even though the OS page size differs (16 KiB and 4 KiB).
- **The 1.0.4 deadlock is not specific to macOS.** `liveness-16` and `liveness-32` fail on Linux with the same messages (0 of 5,240 and 0 of 3,494 tasks drained).
- **Memory follows the same pattern** in resident set size: only v2 gives its memory back after a burst (2,736 KB, against 9,916 KB for v1 and 9,928 KB for 1.0.4).
- **Purge and hardening cost more on the older CPU** (584.7 against 51.1 ns per block in full-pool bursts; 13 ns per task for hardening). On Linux, `MADV_DONTNEED` drops the pages, so each one is faulted in and zeroed again on reuse.
- **Throughput and latency do not separate the versions** here either: 19.0k tasks/s each, limited by 4 hardware threads.

10 Findings

1. **The released 1.0.4 can deadlock.** With PMAD, a 16 B payload shares a size class with `Connection` (16 B), and a 32 B payload with `Worker` (32 B). If the queue fills with such tasks before a worker is connected, the next worker is either disconnected at once (no memory for its `Connection`) or refused at READY ("worker pool full"). The queue can then never be drained until the daemon restarts (Table 4). v1 and v2 pass these tests because of the control pool. A 1.0.5 should either take the same fix or reserve PMAD blocks for connections and workers.
2. **Purge has a burst cost.** When the whole pool is filled and emptied repeatedly, every cycle purges and re-uses 441 pages (about 1 µs each), so a burst costs 102 instead of 8.7 ns per block on the Mac, and 584.7 instead of 51.1 ns on the server. For the daemon, one such full cycle of 8,064 tasks takes far longer than the extra 0.8 ms, so the effect is small, but a workload of exactly this shape would pay for it. Raising `warm` or purging only after a delay, as jemalloc does, would reduce it.

3. **Hardening costs more on an older CPU.** On the M4 Pro the encoded links and checks cost about 1 ns per task (13.8 → 14.8). On the i3-2100 they cost about 13 ns (60.0 → 72.8, 21%) (Appendix A). The daemon's throughput is the same either way, because the allocator is a tiny part of a round trip. PMAD's fixed lists stay the fastest per allocation on both machines (13.3 ns and 43.3 ns); apool pays for its capacity with these extra nanoseconds.
4. **Fullest-first does not help queue traffic** (Section 4) and is off.
5. **Lifetime classes are unused by the daemon.** They fix the pinned case only when a caller passes a hint, and today every task is freed in arrival order.
6. **The reserve costs capacity**: 1.4% at 100 B, and 2.5% of the deepest mixed queue.
7. **Open-loop tail latency varies between runs on both machines.** For the *same* version, p99.9 ranged from 129 to 4,470 µs on the Mac and from 137 to 4,296 µs on the server. The medians of the five runs are close (143–252 µs on the Mac, 207–214 µs on the server), and no version is consistently worse.
8. **The branch reported version 1.0.3** while these runs were made, because it was made before the 1.0.4 version bump. It is 2.0.0 now (`quicx/include/cli.h`).

11 Threats to validity

- **Synthetic load.** The workloads are synthetic: fixed sizes, a random size mix, and nginx lines generated by a program rather than taken from a real log. Real traffic may mix sizes differently.
- **Two machines.** One Mac and one Linux PC. Other CPUs and kernels may differ. On the server the CPU frequency governor (`schedutil`) could not be fixed without root access, so its timings include frequency changes.
- **Shared CPU.** Clients and daemon share one machine over loopback, so they compete for CPU. Absolute throughput is a property of this setup, not of Quicx alone, which is why only the differences between versions are interpreted.
- **Different memory measures.** `phys_footprint` (macOS) and resident set size (Linux) are not the same measure, so memory numbers should be compared within one machine only.
- **v1 is emulated in the allocator benchmark.** In `bench_apool`, "v1" is v2's code with its features off. The end-to-end v1 is the real commit.
- **One thread.** The daemon is single-threaded. None of the published allocators' multi-thread features were needed or tested.

12 Conclusion

Combining published allocator ideas inside a fixed budget made the budget usable. The same 8 MiB holds about 71 times more 100-byte tasks, it follows changes in task size without a restart, and its memory goes back to the operating system after a burst, all without changing throughput or latency, on a 2024 Mac and on a 2011 Linux PC alike. Measuring each idea on its own showed which ones matter for a queue. The reserve and lifetime classes fix specific failure patterns, hardening is cheap safety, purge has a small cost on full-pool bursts, and fullest-first is not worth it. The tests also showed that the released 1.0.4 can deadlock with small tasks, which the control pool fixes.

References

[1] Dimitar Anastasov. PMAD: a percentage-split memory allocator. Part of the Quicx source tree, `pmad/`, 2026. The static allocator used by Quicx up to v1.0.4.

[2] Apple. XNU kernel source, `bsd/sys/mman.h` (`MADV_FREE_REUSABLE`, `MADV_FREE_REUSE`). `https://github.com/apple-oss-distributions/xnu/blob/main/bsd/sys/mman.h`, 2026. Accessed 2026-11-08.

[3] Emery D. Berger, Kathryn S. McKinley, Robert D. Blumofe, and Paul R. Wilson. Hoard: A scalable memory allocator for multithreaded applications. In *Proceedings of the Ninth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-IX)*, pages 117–128, 2000. doi: 10.1145/378993.379232.

[4] Jeff Bonwick. The slab allocator: An object-caching kernel memory allocator. In *USENIX Summer 1994 Technical Conference*, Boston, MA, 1994. URL `https://www.usenix.org/legacy/publications/library/proceedings/bos94/bonwick.html`.

[5] Simon Brown. The C4 model for visualising software architecture: notation, and the software architecture diagram review checklist. `https://c4model.com/diagrams/notation` and `https://c4model.com/diagrams/checklist`, 2026. Accessed 2026-11-09.

[6] Jonathan Corbet, Alessandro Rubini, and Greg Kroah-Hartman. *Linux Device Drivers*. O'Reilly, third edition, 2005. ISBN 0-596-00590-3. Memory pools (`mempool`): chapter 8.

[7] Jason Evans. A scalable concurrent `malloc(3)` implementation for FreeBSD. In *BSDCan*, Ottawa, Canada, 2006. URL `https://people.freebsd.org/~jasone/jemalloc/bsdcan2006/jemalloc.pdf`.

[8] Mel Gorman. *Understanding the Linux Virtual Memory Manager*. Prentice Hall, 2004. ISBN 0-13-145348-3. Zone watermarks: chapter 2.

[9] Daan Leijen, Benjamin Zorn, and Leonardo de Moura. Mimalloc: Free list sharding in action. In *Programming Languages and Systems (APLAS 2019)*, volume 11893 of *Lecture Notes in Computer Science*. Springer, 2019. doi: 10.1007/978-3-030-34175-6_13. Extended version: Microsoft Research technical report MSR-TR-2019-18.

[10] Linux man-pages project. madvise(2) — give advice about use of memory. `https://man7.org/linux/man-pages/man2/madvise.2.html`, 2026. Accessed 2026-11-08.

[11] Martin Maas, David G. Andersen, Michael Isard, Mohammad Mahdi Javanmard, Kathryn S. McKinley, and Colin Raffel. Learning-based memory allocation for C++ server workloads. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '20)*, pages 541–556, 2020. doi: 10.1145/3373376.3378525.

[12] memcached project. Release notes for v1.4.11: slab_reassign and slab_automove. `https://github.com/memcached/memcached/wiki/ReleaseNotes1411`, 2012.

A Server tables

Allocator	100 B	900 B	40 B	300 B	starve	pinned	FIFO	random	ns/task
PMAD, default config	655	180	1,310	262	180	180	5,312	5,312	43.3 (42.5–44.2)
apool v1	47,450	8,143	74,679	22,419	0	0	30,720	30,208	103.3 (103.0–105.1)
apool v2	46,872	8,064	73,584	22,176	16	7,648	29,952	29,184	72.8 (72.7–73.0)
v2 + fullest first	46,872	8,064	73,584	22,176	16	7,648	29,696	29,184	84.9 (84.8–84.9)
v2 − reserve	47,523	8,176	74,606	22,484	0	7,760	30,208	29,696	72.7 (72.7–72.8)
v2 − hardening	46,872	8,064	73,584	22,176	16	7,648	29,952	29,184	60.0 (59.9–61.1)
v2 − lifetime hints	46,872	8,064	73,584	22,176	16	16	29,952	29,184	72.7 (72.6–73.0)
v2 − one block/task	46,848	8,024	73,656	22,101	17	7,616	30,208	29,952	120.5 (120.2–121.8)

Table 12: Allocator-only ablation on the server (`results/server/bench_apool_*.txt`).

Version	100 B	900 B	40 B	300 B	nginx	idle	full	after drain	all drained
1.0.4	655	180	1,310	262	1,433	9,928 (9,924–9,928)	9,928 (9,924–9,928)	9,928 (9,924–9,928)	yes
v1	47,158	8,092	74,240	22,272	31,708	1,784 (1,776–1,784)	9,900 (9,892–9,900)	9,916 (9,908–9,916)	yes
v2	46,593	8,016	73,146	22,044	30,947	1,796 (1,796–1,800)	9,816 (9,816–9,820)	2,736 (2,736–2,740)	yes
v2, all features on	46,593	8,016	73,146	22,044	30,947	1,800 (1,796–1,800)	9,820 (9,816–9,820)	2,740 (2,736–2,740)	yes

Table 13: Whole-daemon capacity and memory on the server. Memory measure: `rss`.

B Reproducing

```
cd docs/report
scripts/build.sh mac            # binaries into build/mac/
```

```
scripts/run_all.sh mac          # raw output into results/mac/ (about 10 minutes)
scripts/run_server.sh           # the same on the server, results copied back
python3 scripts/make_tables.py  # tables/ from results/
scripts/export_figures.sh       # diagrams: SVG, PDF, PNG, outlined SVG
tectonic -X compile main.tex    # this PDF
```